

High-Level Design

DOCUMENT :: 2026-08-06

High-Level Design — CREDENCE (LMS Prototype)

Date: 2026-08-05 Companion: [LLD](#)

1. Purpose

A deployable prototype that demonstrates the full personal-loan lifecycle: application → automated underwriting → disbursement → amortization → EMI payment → balance decrement. Demoable from a single hosted URL in under two minutes using seed data that exercises both origination and mid-repayment flows.

2. Scope

In scope

- One loan product (fixed-rate personal loan, integer cents)
- Two roles: applicant and admin
- Email + password auth (JWT in httpOnly cookie)
- Rule-based underwriting (FOIR + income-multiple) with admin override
- Amortization schedule, installment payment, outstanding balance tracking
- Seeded admin and two applicants (one fresh, one mid-flight) for walkthrough

Out of scope — real KYC, credit bureau, document upload, email/SMS, late fees, prepayment, multi-product, multi-tenancy, audit log, PDF statements, real payments.

3. Architecture

One Node process owns the HTTP port, serves `/api/*` as JSON, and serves the built React bundle for everything else. There is no second service, no reverse proxy, no CORS.

```
flowchart LR
    Browser["Browser<br/>React 19 SPA"]
    Server["Node process :3000<br/>Fastify + built SPA"]
    DB["SQLite file<br/>apps/api/data/lms.db"]

    Browser -- "HTTPS / JSON" --> Server
    Server -- "Drizzle ORM" --> DB
    Server -. "httpOnly cookie<br/>lms_session" .-> Browser
```

Why one process: the brief calls for one live URL. Splitting API + static into two services would add CORS, two deploy targets, and two env surfaces for zero benefit at this scale.

4. Technology stack

Layer	Choice	Rationale
Frontend	React 19, Vite, TypeScript, Tailwind, shadcn	Modern, typesafe, single component model
Routing	React Router 6	Mature, shadcn examples assume it
Server state	TanStack Query	Cache + invalidation without Redux

Layer	Choice	Rationale
Forms	react-hook-form + zodResolver	Uncontrolled, works with shared Zod schemas
Backend	Fastify, TypeScript, Node 20 LTS	Fast, typesafe, plugin model
Validation	Zod	One schema, two consumers (API + web)
ORM	Drizzle	Typesafe queries, plain SQL, easy migrations
Database	SQLite (better-sqlite3)	Zero-config, file-based, fine for one writer
Auth	@fastify/jwt + @fastify/cookie + bcrypt	httpOnly cookie, no CSRF for our routes
Workspace	pnpm workspaces	Single lockfile, no Turborepo overhead
Deploy	Render single Node service	Free tier, persistent disk

5. Data flow — apply to first EMI

```
sequenceDiagram
    participant U as Applicant
    participant W as Web
    participant A as Fastify
    participant D as SQLite

    U->>W: fill /apply form
    W->>A: POST /api/applications
    A->>A: validate (Zod) + compute EMI
    A->>A: run underwriting rule
    A->>D: INSERT application
    A-->>W: 201 application

    Note over W,A: Admin opens /admin queue, reviews, approves + disburses

    W->>A: POST /api/admin/applications/:id/disburse
    A->>D: INSERT loan + installments
    A-->>W: 200 loan + schedule

    U->>W: open /loans/:id, click Pay EMI
    W->>A: POST /api/loans/:id/installments/1/pay
    A->>D: UPDATE installment + decrement outstanding
    A-->>W: 200 paid installment + updated loan
```

6. Data model

Four tables, foreign keys cascading on user delete.

```
erDiagram
    users ||--o{ loan_applications : "submits"
    users ||--o{ loans : "owns"
    loan_applications ||--o{ loans : "becomes"
    loans ||--o{ installments : "has"

    users {
        string id PK
    }
```

```

    string email UK
    string password_hash
    string role
    int    monthly_income_cents
  }
  loan_applications {
    string id PK
    string user_id FK
    int    amount_cents
    int    term_months
    int    annual_rate_bps
    string status
    string decided_by FK
  }
  loans {
    string id PK
    string application_id FK
    string user_id FK
    int    principal_cents
    int    outstanding_cents
    string status
  }
  installments {
    string id PK
    string loan_id FK
    int    sequence
    int    principal_due
    int    interest_due
    int    paid_amount
    string paid_at
  }
}

```

Money is stored as `integer cents` everywhere — no floats. Dates are ISO 8601 strings.

7. Security

Concern	Approach
Passwords	bcrypt cost 10
Sessions	JWT in <code>httpOnly</code> , <code>SameSite=Lax</code> cookie (<code>1ms_session</code> , 1h) + refresh token (<code>1ms_refresh</code> , 30d)
CSRF	Not needed: cookie is <code>SameSite=Lax</code> , no state-changing GETs
XSS	React default escaping + <code>shadcn</code> primitives; no <code>dangerouslySetInnerHTML</code>
SQL injection	Drizzle parameterizes every query
Secrets	<code>JWT_SECRET</code> / <code>JWT_REFRESH_SECRET</code> / <code>DATABASE_URL</code> from env, never in repo
CORS	Not needed (same origin)
Rate limiting	Not implemented (Render edge handles DDoS)
Audit log	Not implemented (out of scope)

8. Deployment

Build: `pnpm install --frozen-lockfile && pnpm build → apps/web/dist/ + apps/api/dist/`.

Start: `pnpm start` runs `node apps/api/dist/server.js`. On boot, the server runs `migrate` then `seed` (idempotent upsert), so a cold start always ends with a usable DB.

Render setup: single Node service, free tier, persistent disk at `/data`. `DATABASE_URL=/data/lms.db`. Pushes to `main` auto-deploy.

Env vars (Render dashboard, mirrored in `.env.example`): `DATABASE_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `NODE_ENV=production`, `PORT=10000`.

9. Observability

Out of scope. Render's request log dashboard is the prototype's only signal. No metrics, no tracing, no structured logging.

10. Future work

Anything in the spec's "out of scope" list: real KYC, real payments, document upload, multi-product, audit log, PDF statements, email notifications, late-fee calculation, prepayment, mobile polish.

End of HLD. For schemas, endpoints, and per-module detail see the [LLD](#).